



UNS
UNIVERSIDAD
NACIONAL DEL SANTA

CURSO:
APLICACIONES MÓVILES

INTERFACES Y MATERIAL DESIGN

Diseña experiencias increíbles con
Android y **Material Design**



Android



Material
Design



Interfaces
Modernas



Experiencia
de Usuario



Desarrollo
Android



Dr. MIRKO MANRIQUE RONCEROS





Objetivo general

Comprender los fundamentos del diseño de interfaces y los principios de Material Design para desarrollar aplicaciones móviles con interfaces **modernas, intuitivas y orientadas a mejorar la experiencia del usuario.**



Objetivos específicos



Identificar los **elementos fundamentales** que componen una interfaz de usuario en aplicaciones móviles.



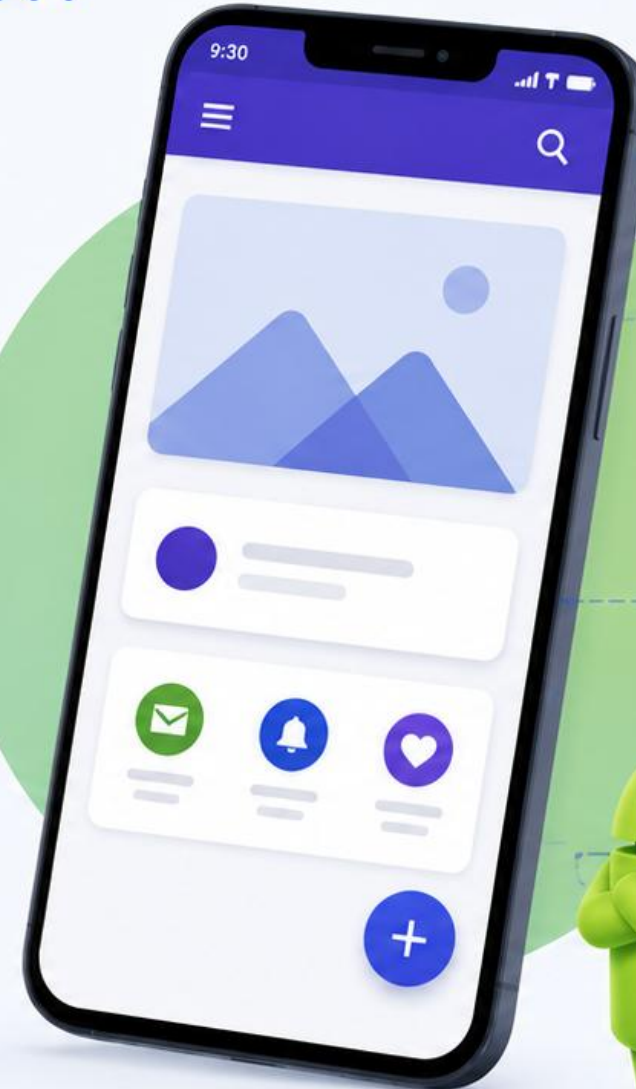
Reconocer la importancia de la **usabilidad** y el diseño visual en el desarrollo de aplicaciones móviles.



Comprender los principios y características de **Material Design** como sistema de diseño para aplicaciones móviles.



Aplicar **componentes de Material Design** en la construcción de interfaces gráficas.



Typography



Color



Elevation



Interfaces modernas



Experiencia intuitiva



Enfoque en el usuario



Desarrollo eficiente

INTERFACES

Una interfaz es el conjunto de **elementos visuales** mediante los cuales **el usuario interactúa** con una aplicación.



Ejemplos:



Botones.

Aa

Campos de texto.



Imágenes.



Menús.



Listas.



Formularios.



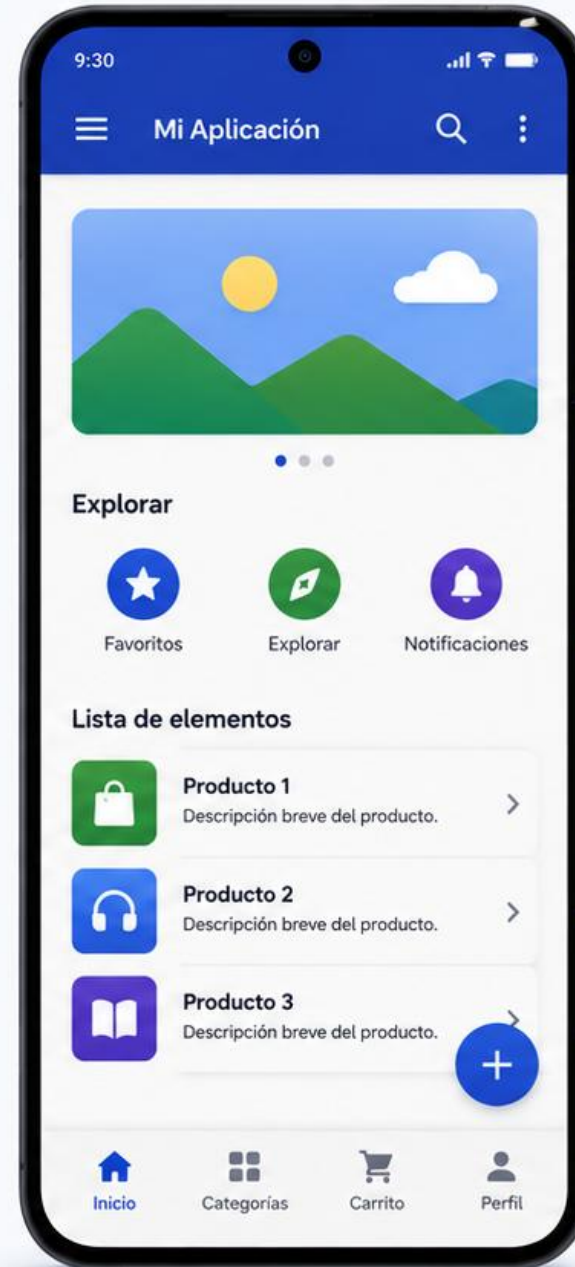
Tarjetas.



Navegación.



Una buena interfaz mejora la **experiencia del usuario**, facilita la navegación y hace que la aplicación sea más **útil, intuitiva y atractiva**.



Iniciar sesión

Botón
Acciones principales.

Tu nombre

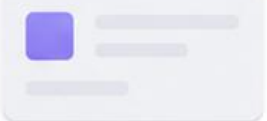
Campo de texto
Entrada de información.



Imagen
Contenido visual.

- Inicio
- Perfil
- Ajustes

Menú
Opciones de navegación.



Tarjeta
Agrupa información.



Objetivos de una interfaz móvil



A Facilitar la interacción



El usuario debe comprender rápidamente:

- ✓ Qué puede hacer.
- ✓ Cómo hacerlo.
- ✓ Qué información visualizar.

B Mejorar la experiencia del usuario



Una buena interfaz:

- ✓ Reduce errores.
- ✓ Incrementa productividad.
- ✓ Mejora satisfacción.

C Organizar la información



La interfaz debe:

- ✓ Mostrar información jerárquicamente.
- ✓ Priorizar contenido importante.

D Adaptarse a diferentes dispositivos



Debe funcionar correctamente en:

- ✓ Smartphones.
- ✓ Tablets.
- ✓ Foldables.
- ✓ Diferentes resoluciones.



Interacción intuitiva



Experiencia positiva



Información ordenada

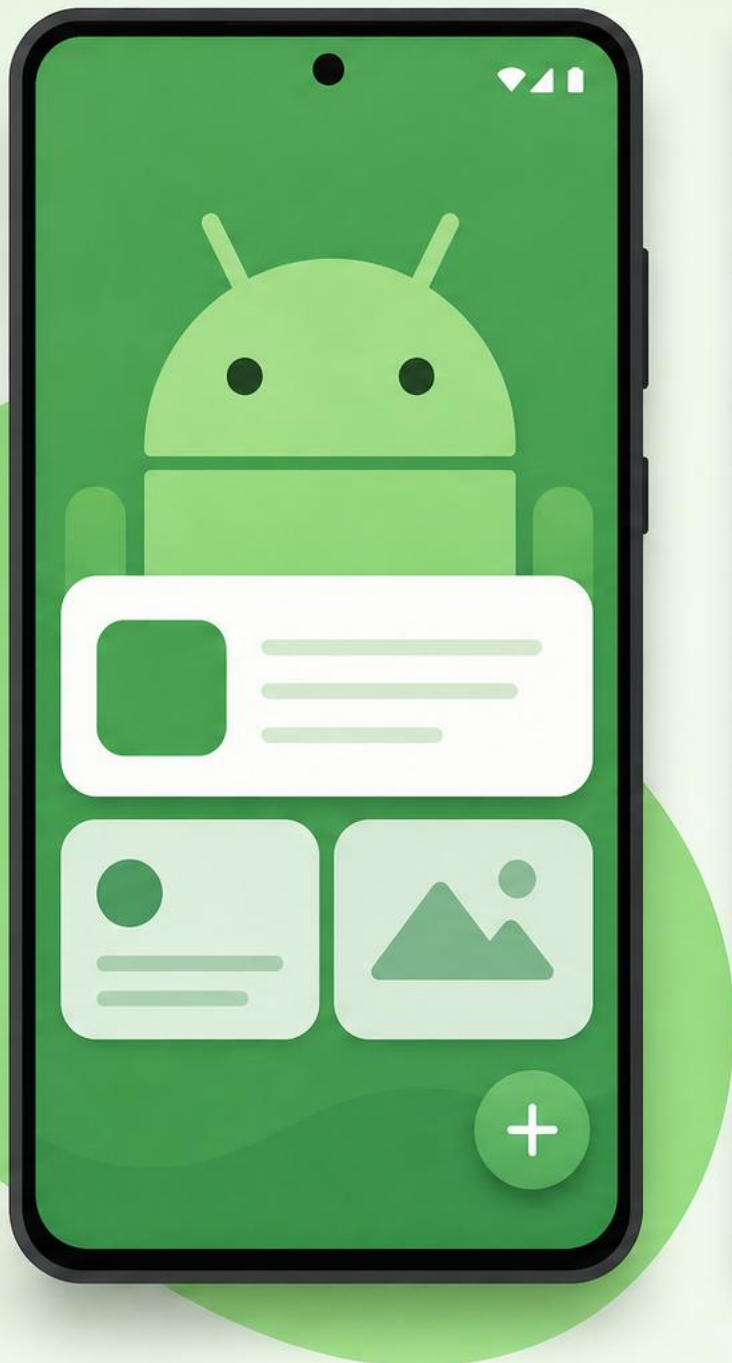


Diseño adaptativo



Una interfaz bien diseñada conecta al usuario con lo que realmente importa: contenido, funcionalidades y experiencias significativas.





Android utiliza una arquitectura basada en:



Activity



Layout XML



Views y ViewGroups



Activity y la interfaz



Una **Activity** representa una pantalla de la aplicación.



La interfaz visual normalmente se define en archivos **XML**.



```
activity_main.xml
<?xml version="1.0"
    encoding="utf-8"?>
<LinearLayout xmlns:android=
    "http://schemas.android.com/
    apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
    <ImageView
        android:layout_width="match_parent"
        android:layout_height="180dp"
        android:src="@drawable/image" />
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Título principal"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_marginTop="16dp" />
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Este es un ejemplo de
        una interfaz definida en XML."
        android:layout_marginTop="8dp" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="ACCIONAR"
        android:layout_marginTop="16dp" />
</LinearLayout>
```

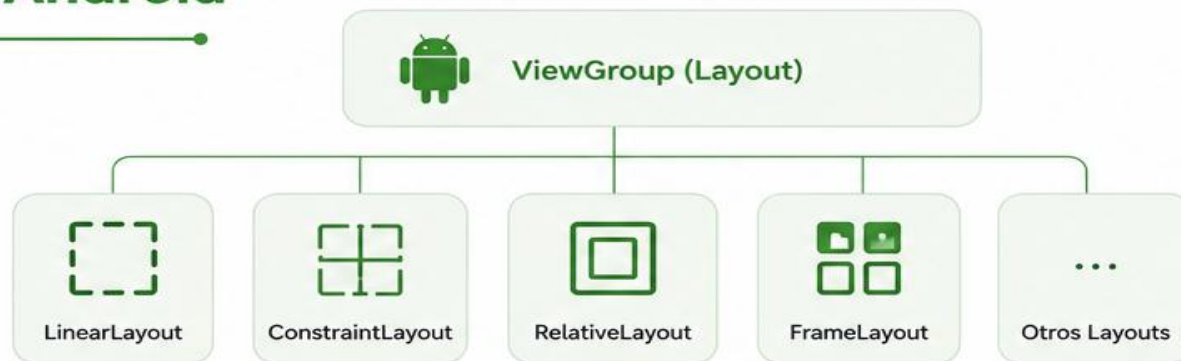


Layouts en Android

Los **Layouts** son contenedores que organizan componentes visuales.

Android los clasifica como:

- ✓ ViewGroup.
- ✓ Contenedores de UI.





Tipos principales de Layouts



Los **Layouts** en Android son contenedores que organizan y estructuran los componentes visuales en la interfaz de usuario.

A LinearLayout



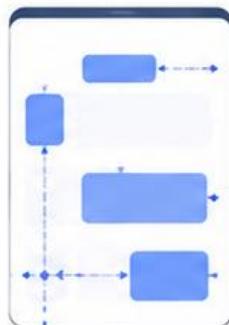
Organiza elementos:

- Verticalmente.
- Horizontalmente.

Ventajas

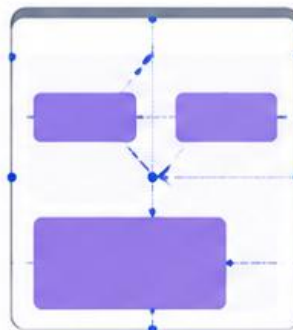
- Fácil implementación.
- Ideal para formularios simples.

B RelativeLayout



Permite posicionar componentes respecto a otros.

C ConstraintLayout

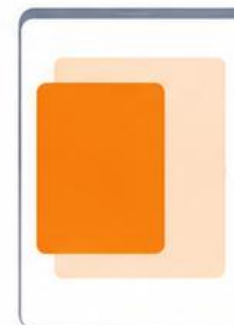


Es el layout moderno recomendado por Google.

Permite:

- Diseños complejos.
- Interfaces responsivas.
- Menor anidamiento.

D FrameLayout



Contenedor para superponer elementos.



Uso común:

- Fragments.
- Pantallas dinámicas.

E ScrollView



Permite desplazamiento vertical.



Tip

Elige el layout adecuado según las necesidades de tu interfaz para obtener apps más eficientes, escalables y fáciles de mantener.



Buenas prácticas:

Usa ConstraintLayout para nuevos proyectos siempre que sea posible. Evita anidar demasiados layouts para mejorar el rendimiento.



Views

(Componentes visuales)

- Las Views son elementos individuales de interfaz.

A

Tt

TextView

Muestra texto.

Ejemplo:
¡Bienvenido a
Material Design!

B



EditText

Permite ingresar texto.

Ejemplo:
Escribe aquí...

C

BUTTON

Button

Ejecuta acciones

Ejemplo:

ENVIAR

10:30

Mi Aplicación

¡Bienvenido a Material Design!

Escribe aquí...

ENVIAR



Lista de elementos

- Elemento 1
Descripción del elemento
- Elemento 2
Descripción del elemento
- Elemento 3
Descripción del elemento

Tarjeta de ejemplo

Esta es una tarjeta creada con
CardView.



D



ImageView

Muestra imágenes.

Ejemplo:



E



RecyclerView

Lista moderna
y eficiente

Ejemplo:



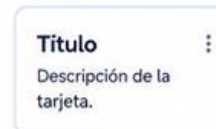
F



CardView

Permite crear
tarjetas visuales.

Ejemplo:



G



FloatingActionButton

Botón flotante
Material Design

Ejemplo:



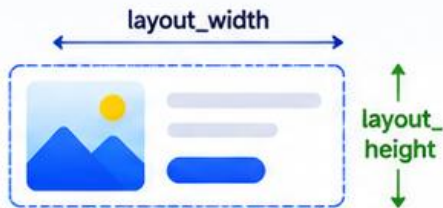
Las Views son la base para construir interfaces ricas, intuitivas y adaptadas a las necesidades del usuario.



Atributos importantes de INTERFAZ

A layout_width y layout_height

Definen tamaño.



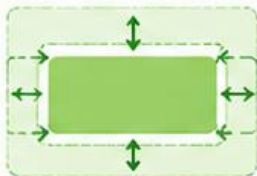
Valores comunes:

- ✓ match_parent
- ✓ wrap_content
- ✓ dp

B margin y padding

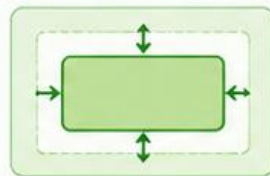
Margin

Espacio externo.



Padding

Espacio interno.



¡Importante!

Estos atributos permiten crear interfaces **organizadas**, **adaptables** y visualmente **consistentes** en Android.



```
<TextView
    android:id="@+id/titulo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_margin="16dp"
    android:padding="8dp"
    android:gravity="center"
    android:textSize="18sp"
    android:textColor="#FFFFFF" />
```

C gravity y layout_gravity

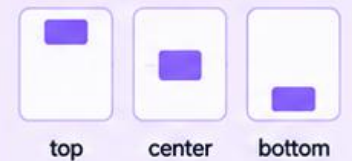
gravity

Alinea contenido interno.



layout_gravity

Alinea la vista dentro del contenedor.



D id

Identificador único



E background

Color o imagen de fondo



F Unidades de medida

dp (Density-independent pixels)

Unidad recomendada para tamaños.



G sp (Scale-independent pixels)

Usada para texto.

Aa





Interfaz y UX

Una buena interfaz debe ser:

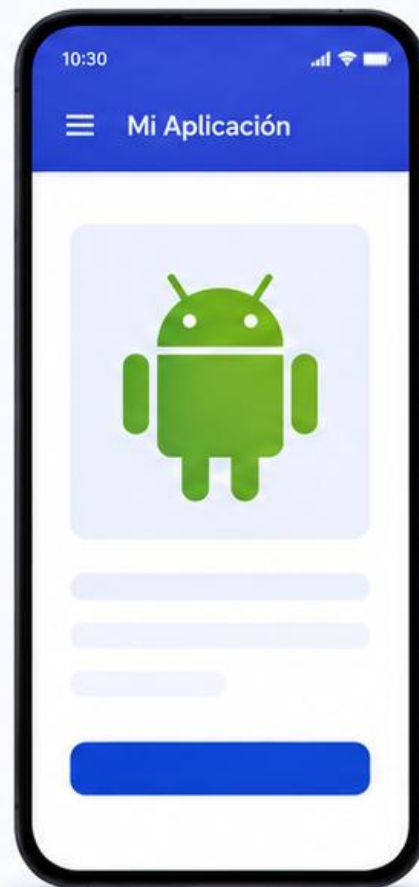
Característica	Descripción
 Intuitiva	Fácil de comprender
 Consistente	Diseño uniforme
 Accesible	Para todos los usuarios
 Responsiva	Adaptable
 Minimalista	Sin sobrecarga





Ciclo de vida e interfaz

Las interfaces dependen del ciclo de vida de **Activities** y **Fragments**.



```
<LinearLayout
  xmlns:android="
    http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:padding="16dp">

  <!-- Contenido de la interfaz -->
</LinearLayout>
```



Eventos importantes:



onCreate()

Se crea la actividad o fragmento. Aquí se inicializa la interfaz.



onStart()

La actividad se vuelve visible para el usuario.



onResume()

La actividad está en primer plano y el usuario puede interactuar con ella.



onPause()

La actividad pierde el foco, pero aún puede ser visible parcialmente.



onDestroy()

La actividad o fragmento se destruye y se liberan los recursos.



Importante: Manejar correctamente el ciclo de vida permite mantener una interfaz **eficiente**, **evitar fugas de memoria** y **mejorar la experiencia del usuario**.





Ciclo de vida e interfaz

Las interfaces dependen del ciclo de vida de **Activities** y **Fragments**.



```
<LinearLayout
  xmlns:android="
    http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:padding="16dp">

  <!-- Contenido de la interfaz -->
</LinearLayout>
```



Eventos importantes:



onCreate()

Se crea la actividad o fragmento. Aquí se inicializa la interfaz.



onStart()

La actividad se vuelve visible para el usuario.



onResume()

La actividad está en primer plano y el usuario puede interactuar con ella.



onPause()

La actividad pierde el foco, pero aún puede ser visible parcialmente.



onDestroy()

La actividad o fragmento se destruye y se liberan los recursos.



Importante: Manejar correctamente el ciclo de vida permite mantener una interfaz **eficiente**, **evitar fugas de memoria** y **mejorar la experiencia del usuario**.





Conclusiones



Las **interfaces** son fundamentales en aplicaciones móviles.



Android proporciona múltiples **herramientas** para diseñarlas.



ConstraintLayout y **Material Design** son estándares modernos.



Una buena interfaz **mejora** significativamente la experiencia del usuario.



Jetpack Compose representa el futuro del diseño de interfaces Android.



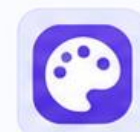
Material Design



ConstraintLayout



Layouts



Material Design

```
@Composable
fun TarjetaEjemplo() {
    Card(
        modifier = Modifier.padding(16.dp),
        elevation = CardDefaults.cardElevation()
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Diseña mejores experiencias",
                style = MaterialTheme.typography.titleMedium)
            Text("Usa herramientas modernas para crear interfaces increíbles.")
        }
    }
}
```





MATERIAL DESIGN

Material Design es un **sistema de diseño visual** creado por Google para desarrollar interfaces **modernas, intuitivas, consistentes y adaptables** en aplicaciones móviles, web y multiplataforma.



Fue presentado oficialmente en **2014** y posteriormente evolucionó hacia:



Material Design 2



Material You
(Material Design 3)



Actualmente, **Android** utiliza principalmente **Material Design 3**, el cual incorpora:



Personalización
dinámica.



Adaptabilidad.



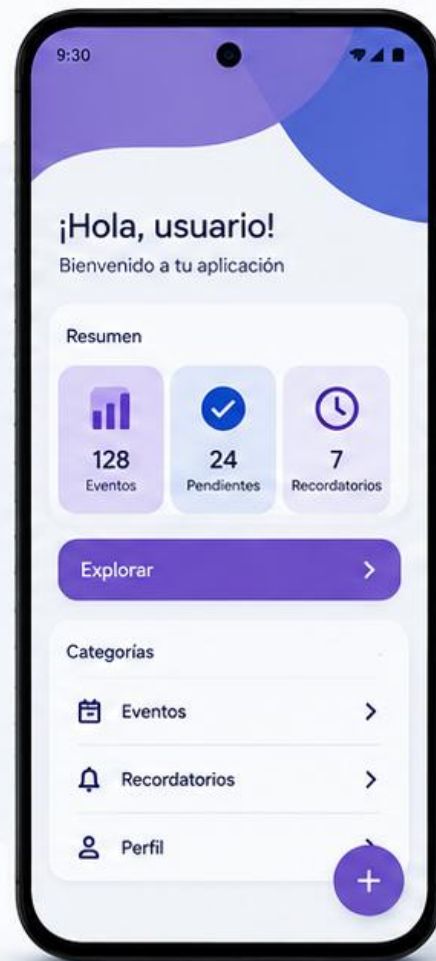
Mejor
accesibilidad.



Diseño
responsivo.



Experiencia
visual moderna.



Material Design 3 potencia la creación de **experiencias más humanas**, útiles y atractivas, alineadas con las necesidades del usuario actual.





Material Design es una guía de diseño que define:



Cómo deben verse las interfaces.



Cómo deben comportarse los componentes.



Cómo interactúa el usuario con la aplicación.



Cómo mantener consistencia visual y experiencia de usuario.

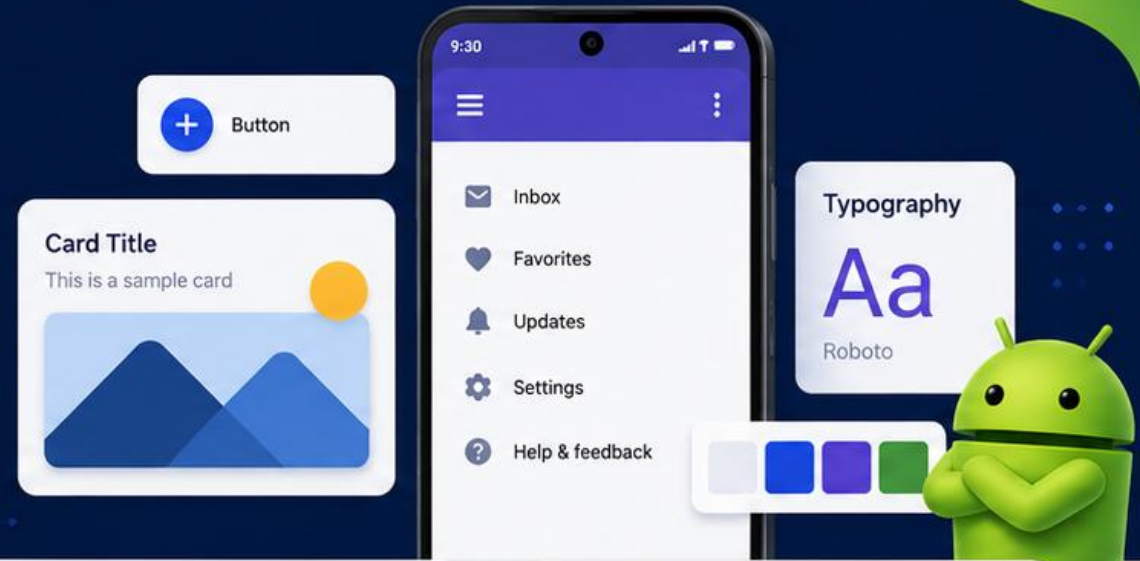


“ Google lo define como:
“Un sistema adaptable de directrices, componentes y herramientas que respaldan las mejores prácticas de diseño de interfaces de usuario.”

”

Objetivos de Material Design

Material Design tiene como propósito crear experiencias digitales más humanas, coherentes y eficientes a través de principios de diseño claros y componentes reutilizables.



A) Mejorar la experiencia del usuario (UX)

Busca que las aplicaciones sean:

- ✓ Fáciles de usar.
- ✓ Intuitivas.
- ✓ Visualmente agradables.
- ✓ Accesibles.



B) Mantener consistencia

Todas las aplicaciones Android deben compartir:

- ✓ Estilos visuales.
- ✓ Comportamientos similares.
- ✓ Navegación uniforme.



Esto reduce la curva de aprendizaje del usuario.



C) Facilitar el desarrollo

Material Design proporciona:

- ✓ Componentes prediseñados.
- ✓ Guías oficiales.
- ✓ Librerías listas para usar.



D) Adaptabilidad

Permite que la interfaz:

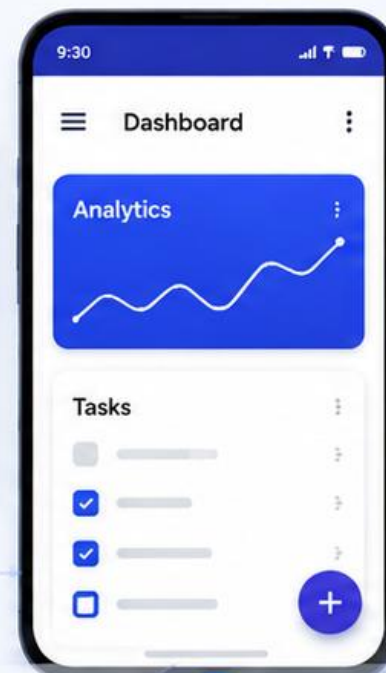
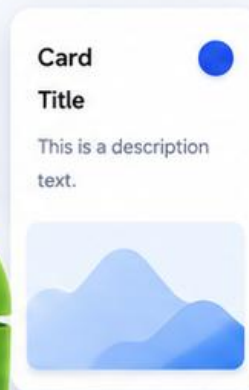
- ✓ Se adapte a distintos tamaños de pantalla.
- ✓ Funcione en teléfonos, tablets y foldables.
- ✓ Sea responsiva.



Material Design no es solo una guía de estilo, es una filosofía centrada en el usuario, la coherencia y la eficiencia.

Principios Fundamentales de Material Design

Material Design se basa en principios que guían la creación de interfaces **modernas, intuitivas y consistentes**.



A Material como metáfora

La interfaz se comporta como si estuviera hecha de:

- Papel.
- Tinta.
- Superficies físicas.



B Movimiento significativo

Las animaciones:

- ✓ No son decorativas.
- ✓ Ayudan al usuario a comprender transiciones.



C Diseño adaptable

La interfaz debe:

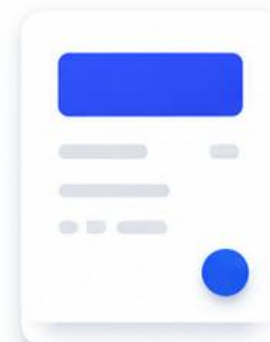
- ✓ Ajustarse automáticamente.
- ✓ Mantener coherencia en cualquier dispositivo



D Jerarquía visual

Se utiliza:

- Aa Tamaño.
- Color.
- Elevación.
- Espaciado.



Estos principios trabajan juntos para crear **experiencias coherentes, claras y útiles**, centradas en el usuario.



Componentes Principales de Material Design

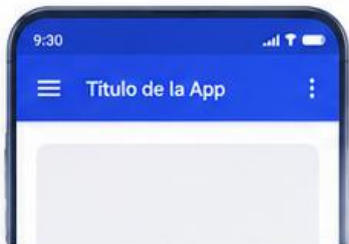
Material Design proporciona un conjunto de componentes reutilizables que siguen principios de diseño consistentes para crear experiencias modernas, intuitivas y eficientes.



A App Bar (Toolbar)

Barra superior que contiene:

- ✓ Título.
- ✓ Navegación.
- ✓ Acciones.



B Floating Action Button (FAB)

Botón flotante circular para acciones principales.

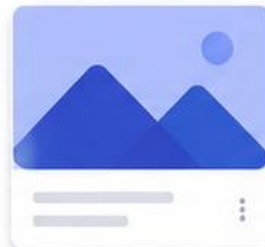


C Cards (Tarjetas)

Permiten agrupar información visualmente.

Usos:

- ✓ Noticias.
- ✓ Productos.
- ✓ Eventos.

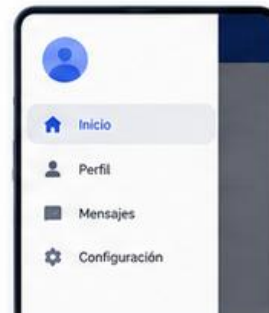


D Navigation Drawer

Menú lateral de navegación.

Usado para:

- ✓ Navegar entre secciones.

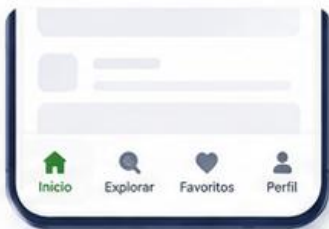


E Bottom Navigation

Barra inferior para navegación rápida.

Ideal para:

- ✓ Apps con 3 a 5 secciones principales.



F Snackbar

Mensaje temporal mostrado en la parte inferior.



G TextInputLayout

Campos de texto modernos.

Incluyen:

- ✓ Validación.
- ✓ Etiquetas flotantes.
- ✓ Iconos.

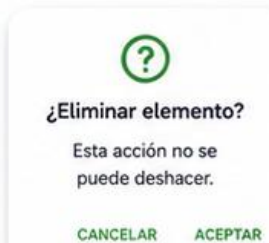


La contraseña es obligatoria

H Dialogs

Ventanas emergentes para:

- ✓ Confirmaciones.
- ✓ Alertas.
- ✓ Formularios.



Estos componentes trabajan juntos para crear interfaces coherentes, accesibles y centradas en el usuario, mejorando la usabilidad y la experiencia general de la aplicación.



Ventaja	Descripción
Consistencia	Interfaces uniformes
UX mejorada	Mayor facilidad de uso
Adaptabilidad	Responsive
Desarrollo rápido	Componentes listos
Accesibilidad	Inclusión
Modernidad	Interfaces profesionales



UI Tradicional	Material Design
 Interfaces rígidas	 Interfaces dinámicas
 Poco movimiento	 Animaciones fluidas
 Menor accesibilidad	 Alta accesibilidad
 Diseño inconsistente	 Diseño uniforme



Conclusiones

- ✓ Material Design es el **estándar moderno** de diseño Android.
- ★ Mejora significativamente la **experiencia del usuario**.
- </> Facilita el **desarrollo profesional** de aplicaciones móviles.
- 👤 Material Design 3 incorpora **personalización dinámica** y **accesibilidad avanzada**.
- 🎓 Es fundamental para cualquier **desarrollador Android moderno** dominar:



Material
Components



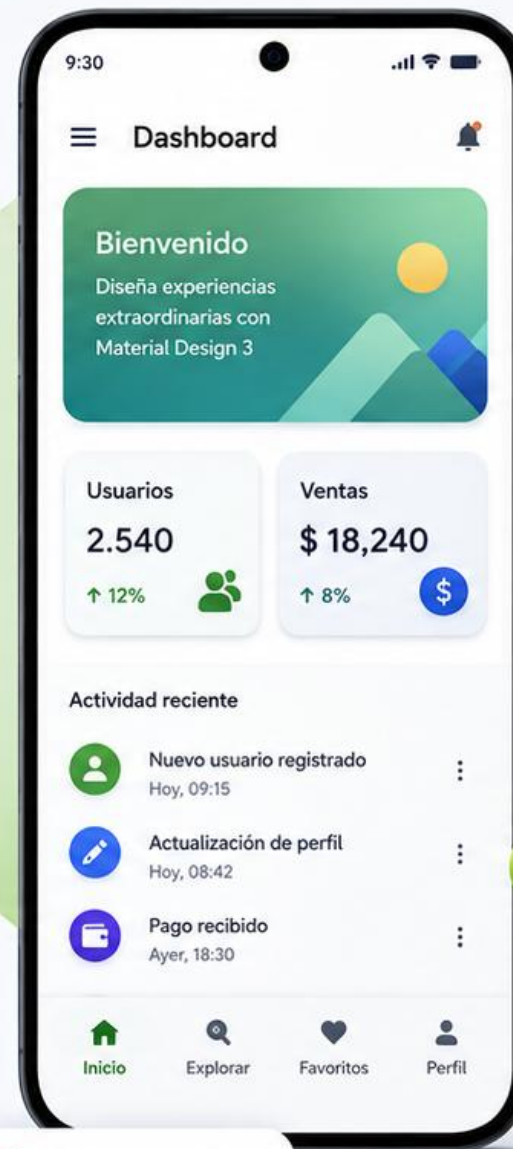
Material
You



Jetpack
Compose



Responsive
Design



Aa



Material
Design 3